

Sistemas de Tiempo Real

Práctica 2: FreeRTOS.

Ingeniería en Computación - 2026

Ejercicio 3

Investigue y analice las herramientas que provee FreeRTOS para la sincronización de tareas.

1. Introducción

En un sistema operativo de tiempo real (RTOS) como FreeRTOS, múltiples tareas se ejecutan de forma concurrente compartiendo recursos limitados: memoria, periféricos, variables globales, buses de comunicación, etc. Cuando dos o más tareas acceden a un mismo recurso, o cuando una tarea depende de un evento generado por otra (o por una interrupción), surge la necesidad de **sincronizar** su ejecución para evitar condiciones de carrera (*race conditions*), corrupción de datos y comportamientos no determinísticos.

FreeRTOS provee un conjunto de mecanismos de sincronización y comunicación entre tareas (y entre tareas e interrupciones) que pueden agruparse en dos grandes finalidades:

- **Exclusión mutua:** garantizar que solo una tarea a la vez acceda a un recurso compartido (sección crítica).
- **Sincronización por eventos / señalización:** permitir que una tarea espere a que ocurra un evento generado por otra tarea o por una ISR (rutina de interrupción).

2. Secciones críticas y suspensión del planificador

Antes de los mecanismos de alto nivel, FreeRTOS ofrece herramientas básicas para proteger código de muy corta duración:

- **Secciones críticas** (`taskENTER_CRITICAL()` / `taskEXIT_CRITICAL()`): deshabilitan las interrupciones (hasta un nivel configurable) durante un tramo muy breve de código, impidiendo cualquier cambio de contexto. Se usan solo para operaciones extremadamente cortas, ya que bloquean todo el sistema.
- **Suspensión del planificador** (`vTaskSuspendAll()` / `xTaskResumeAll()`): impide los cambios de contexto entre tareas pero permite que las interrupciones sigan atendiéndose. Es más permisivo que la sección crítica, aunque tampoco es recomendable para bloques largos.

Estas herramientas son la base, pero para sincronización estructurada entre tareas FreeRTOS recomienda usar los objetos de kernel que se describen a continuación, ya que son más eficientes y seguros.

3. Semáforos

Los semáforos son objetos del kernel implementados sobre la misma estructura de datos que las colas (queues). FreeRTOS distingue tres tipos principales:

3.1 Semáforo binario (Binary Semaphore)

Puede tomar solo dos estados: disponible (1) o no disponible (0). Se utiliza típicamente para señalización entre una ISR y una tarea, o entre dos tareas, donde una tarea queda bloqueada en `xSemaphoreTake()` esperando que otra (o una interrupción) ejecute `xSemaphoreGive()`. A diferencia de un mutex, el semáforo binario no tiene noción de "dueño" ni implementa herencia de prioridad.

```
SemaphoreHandle_t xSemaforo;
xSemaforo = xSemaphoreCreateBinary();

// En la ISR o tarea emisora
xSemaphoreGiveFromISR(xSemaforo, &xHigherPriorityTaskWoken);

// En la tarea receptora
if (xSemaphoreTake(xSemaforo, portMAX_DELAY) == pdTRUE) {
    // recurso/evento disponible
}
```

3.2 Semáforo contador (Counting Semaphore)

Mantiene un contador que puede tomar valores entre 0 y un máximo definido. Se utiliza para contar eventos (por ejemplo, cuántos pulsos generó un sensor) o para gestionar un *pool* de recursos idénticos (por ejemplo, N buffers disponibles). Cada `xSemaphoreGive()` incrementa el contador y cada `xSemaphoreTake()` lo decrementa, bloqueando la tarea si el contador llega a cero.

```
xSemaforoContador = xSemaphoreCreateCounting(MAX_COUNT, INITIAL_COUNT);
```

3.3 Mutex (Mutual Exclusion Semaphore)

Es un semáforo binario especializado en la protección de recursos compartidos (exclusión mutua) en lugar de señalización de eventos. Su diferencia clave es que implementa **herencia de prioridad** (*priority inheritance*): si una tarea de baja prioridad posee el mutex y una tarea de alta prioridad intenta tomarlo, la prioridad de la tarea de baja prioridad se eleva temporalmente hasta la de la tarea de mayor prioridad que espera, evitando así el problema de **inversión de prioridades**. Además, el mutex tiene noción de "dueño": solo la tarea que lo tomó puede liberarlo.

```
SemaphoreHandle_t xMutex;
xMutex = xSemaphoreCreateMutex();

if (xSemaphoreTake(xMutex, portMAX_DELAY) == pdTRUE) {
    // sección crítica: acceso exclusivo al recurso compartido
    xSemaphoreGive(xMutex);
}
```

```
}

QueueHandle_t xCola = xQueueCreate(5, sizeof(uint16_t));

// Tarea productora
xQueueSend(xCola, &valor, portMAX_DELAY);

// Tarea consumidora
if (xQueueReceive(xCola, &valorRecibido, portMAX_DELAY) == pdTRUE) {
    // procesar valorRecibido
}
```

5. Notificaciones de Tareas (Task Notifications)

Son un mecanismo más liviano y rápido introducido para reemplazar, en muchos casos, a los semáforos binarios, contadores y colas de un solo elemento. Cada tarea posee una estructura de notificación integrada (no requiere crear un objeto adicional), lo cual reduce el uso de RAM y el tiempo de ejecución. Se utiliza con las funciones `xTaskNotify()` / `xTaskNotifyFromISR()` (para enviar) y `xTaskNotifyWait()` / `ulTaskNotifyTake()` (para recibir/esperar). Permiten, además, enviar un valor de 32 bits junto con la notificación y elegir cómo se combina (sobrescribir, sumar, hacer OR a nivel de bits, etc.), lo que las hace útiles también como semáforos contadores o incluso como banderas de eventos simplificadas.

```
// Emisor
xTaskNotifyGive(xTareaDestino);

// Receptor
ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
```

Ventaja principal: según la documentación de FreeRTOS, notificar a una tarea puede ser hasta un orden de magnitud más rápido que dar un semáforo, y usan menos memoria RAM. **Limitación:** solo puede haber un "receptor" por notificación (no son adecuadas cuando varias tareas deben esperar el mismo evento de forma independiente, como sí lo permiten los semáforos o los event groups).

6. Grupos de Eventos (Event Groups)

Una *Event Group* es un conjunto de banderas (bits) dentro de una misma variable, donde cada bit representa un evento distinto. Permiten que una tarea se bloquee esperando que se cumpla **una combinación** de eventos (AND u OR de varios bits), y que se sincronicen varias tareas entre sí antes de continuar (por ejemplo, un patrón de "rendezvous" o barrera). Se manipulan con `xEventGroupSetBits()`, `xEventGroupClearBits()` y `xEventGroupWaitBits()`.

```
EventGroupHandle_t xEventGroup = xEventGroupCreate();

// Tarea que setea el evento
xEventGroupSetBits(xEventGroup, BIT_0);

// Tarea que espera BIT_0 Y BIT_1
xEventGroupWaitBits(xEventGroup, BIT_0 | BIT_1,
    pdTRUE,      // limpiar bits al salir
    pdTRUE,      // esperar todos (AND)
    portMAX_DELAY);
```

7. Cuadro comparativo

Herramienta	Uso principal	¿Dueño / herencia de prioridad?	Datos asociados
Sección crítica	Proteger código muy corto (deshabilita interrupciones)	No aplica	Ninguno
Semáforo binario	Señalización simple tarea-tarea o ISR-tarea	No	Ninguno
Semáforo contador	Contar eventos / administrar pool de recursos	No	Contador entero
Mutex	Exclusión mutua sobre recursos compartidos	Sí (herencia de prioridad)	Ninguno
Mutex recursivo	Exclusión mutua con toma anidada por la misma tarea	Sí	Contador de anidamiento
Cola (Queue)	Transferir datos entre tareas/ISR con bloqueo automático	No	Estructuras de datos completas
Notificación de tarea	Señalización rápida y liviana hacia una única tarea	No	Valor de 32 bits (opcional)
Event Group	Sincronizar según combinación de múltiples eventos (AND/OR)	No	Conjunto de banderas (bits)

Inversión de prioridades: ocurre cuando una tarea de alta prioridad queda bloqueada esperando un recurso que posee una tarea de baja prioridad, y una tarea de prioridad intermedia (que no necesita el recurso) impide que la tarea de baja prioridad se ejecute y libere el recurso. El mutex de FreeRTOS resuelve este problema mediante herencia de prioridad; los semáforos binarios/contadores *no* lo resuelven, por lo que no deben usarse para proteger recursos compartidos cuando existe riesgo de este problema.

8. Criterios para elegir la herramienta adecuada

- ¿Necesito proteger un **recurso compartido** (memoria, periférico)? → **Mutex**.
- ¿Necesito **señalizar un evento único** desde una ISR o tarea hacia otra tarea? → **Semáforo binario** o, preferentemente, **notificación de tarea** (más eficiente).
- ¿Necesito **contar ocurrencias** de un evento o administrar un pool de N recursos idénticos? → **Semáforo contador**.
- ¿Necesito **transferir datos** (no solo señalar) entre tareas? → **Cola**.
- ¿Necesito que una tarea espere **una combinación de varios eventos**, o sincronizar varias tareas entre sí (barrera)? → **Event Group**.
- ¿La sincronización es hacia una **única tarea** y se busca máxima velocidad y mínimo uso de RAM? → **Notificación de tarea**.

Conclusión: FreeRTOS ofrece un abanico de herramientas de sincronización que van desde mecanismos de bajo nivel (secciones críticas, suspensión del planificador) hasta objetos de kernel de alto nivel (semáforos, mutex, colas, notificaciones y grupos de eventos). La elección de la herramienta correcta depende del tipo de problema a resolver: exclusión mutua sobre un recurso compartido, señalización de eventos puntuales, transferencia de datos entre tareas, o coordinación basada en múltiples condiciones. Un uso adecuado de estos mecanismos es fundamental para lograr un sistema de tiempo real determinístico, evitar condiciones de carrera y prevenir problemas como la inversión de prioridades.

9. Fuentes

- FreeRTOS.org — Documentación oficial: "Queues, Mutexes and Semaphores", "Task Notifications" y "Event Groups" (www.freertos.org).
- Richard Barry, "Mastering the FreeRTOS Real Time Kernel" — Guía oficial del kernel FreeRTOS.